Linux for Cybersecurity

A beginner command-line handbook with episode-by-episode study notes

Personal study and revision reference

Contents

Linux for Cybersecurity 3

Linux Command Reference 3

Help and Terminal Workflow 5

Files and Directories 6

Users and Permissions 7

Processes and Services 9

Package Management 10

Networking Commands 11

Episode 1: Getting Started with Linux 12

Episode 2: The Linux Filesystem 13

Episode 3: Help, Paths, and Basic File Operations 14

Episode 4: Users, Groups, and sudo 16

Episode 5: Linux Package Management 17

Episode 6: Services and systemd 18

Episode 7: Process Management 20

Episode 8: A Local Web Server and curl 21

Episode 9: Terminal Efficiency 22

Episode 10: File Operations and Failure Modes 23

Final Revision Checklist 25

Linux for Cybersecurity

Beginner Linux study notes for safe command-line practice and basic host review. The section contains topic references and a separate set of notes for every video in NetworkChuck's Linux for Hackers playlist.

Start here

- [Episode-by-episode index](episodes/README.md) - all 10 playlist videos, each linked to its original video and an individual note.
- [Command reference](linux-commands.md)
- [Help and terminal workflow](terminal-workflow.md)
- [Files and directories](files-and-directories.md)
- [Users and permissions](users-and-permissions.md)
- [Processes and services](processes-and-services.md)
- [Package management](package-management.md)
- [Networking commands](networking-commands.md)
- [Final revision checklist](revision-checklist.md)
- [PDF handbook](linux-for-cybersecurity.pdf)

Learning status

The learner has confirmed completing the playlist in conversation. That confirmation records course progress; it does not claim completion of every optional lab or independent exercise. The episode notes are study references, not lab-result records.

Sources and safety

The public playlist feed lists 10 videos. Video descriptions and chapters give direct topic evidence for many episodes, but a complete transcript was not available consistently. Each episode page identifies the evidence available and links supporting manuals. Commands use Debian/Kali examples where relevant. Inspect before changing state, use a disposable VM for practice, and do not test systems without authorization.

Commands that inspect state are distinguished from commands that write files, install packages, change identity, start or stop services, or terminate processes. Example behavior is described without presenting unrun output as a test result.

Linux Command Reference

Use commands with the permissions needed for the task only. Arguments beginning with `-` are options; quoting protects paths containing spaces from shell word splitting.

Inspect the current location

Command	Purpose	Safe example
`pwd`	Print the current working directory.	`pwd`
`whoami`	Print the effective user name.	`whoami`
`id`	Show the effective identity and group memberships.	`id`
`date -Is`	Show the current time in ISO-like format.	`date -Is`

These commands inspect state and do not change files. Output depends on the host and account.

List and search

Command	Purpose and arguments	Safe example
`ls -la PATH`	List entries; `-l` uses a detailed listing and `-a` includes dotfiles.	`ls -la "$HOME"`
`find PATH -type f -name PATTERN`	Search recursively; `-type f` selects files and `-name` matches a name pattern.	`find "$HOME/Documents" -type f -name '*.log'`
`grep -n 'TEXT' FILE`	Search file text; `-n` includes line numbers.	`grep -n 'failed' auth.log`
`wc -l FILE`	Count newline-delimited lines.	`wc -l auth.log`

Searching a broad tree can take time and may produce permission errors. Search only paths you are authorized to inspect.

Read files

Command	Purpose	Safe example
`cat FILE`	Write the file contents to standard output.	`cat notes.txt`
`less FILE`	Read a file interactively; press q to quit.	`less /var/log/syslog`
`head -n N FILE`	Show the first N lines.	`head -n 20 auth.log`
`tail -n N FILE`	Show the last N lines.	`tail -n 20 auth.log`
`tail -f FILE`	Follow appended data until interrupted with Ctrl+C.	`tail -f app.log`

Log locations and access permissions vary by distribution. Do not copy sensitive log data into an unapproved location.

Pipes and redirection

Syntax	Effect	Example		
`A | B`	Send A's standard output to B's standard input.	`grep 'failed' auth.log | wc -l`		
`A > FILE`	Replace FILE with A's standard output.	`printf '%s\n' 'review' > review.txt`		
`A >> FILE`	Append standard output to FILE.	`date -Is >> review.txt`		
`A 2> FILE`	Redirect standard error.	`find /var/log -type f 2> find-errors.txt`		
`A && B`	Run B only if A exits successfully.	`mkdir review && cd review`		

`>` overwrites its destination. Verify the path before using redirection. A pipeline generally passes output, not the first command's success status, unless shell options are configured.

Change or create files

Commands that modify the filesystem are marked **Modify**. Check paths before acting.

Command	Purpose and arguments	Safe example
`mkdir -p PATH`	**Modify:** create directories; `-p` also creates missing parents and does not fail if the directory exists.	`mkdir -p "$HOME/lab-notes"`

Command	Purpose and arguments	Safe example
`touch FILE`	**Modify:** create an empty file or update timestamps.	`touch "$HOME/lab-notes/todo.txt"`
`cp SOURCE DEST`	**Modify:** copy a file.	`cp notes.txt notes.backup.txt`
`mv SOURCE DEST`	**Modify:** move or rename; an existing destination may be replaced depending on options and prompting configuration.	`mv draft.txt draft-reviewed.txt`
`rm FILE`	**Destructive:** remove a directory entry; there is usually no simple undo.	Avoid in routine study examples; confirm exact target and backups first.

Avoid recursive deletion options during exploration. Shell wildcards can expand to more files than intended. Inspect the exact target with `printf '%s\n' PATH` or `ls` before any removal.

Revision points

- `pwd`, `ls`, and `cd` are the core navigation commands highlighted in the accessible episode 1 chapters.
- Inspect before changing; quote variable paths; check permissions and scope.
- `>` overwrites, `>>` appends, and pipes connect command output.
- Treat `rm`, recursive options, and privileged commands as high-impact operations.

References

- NetworkChuck Linux for Hackers playlist
- Episode 1 with chapter markers
- Coreutils command index
- grep manual

Help and Terminal Workflow

The playlist feed includes an episode titled "HELP!! (for when you suck at Linux)" and a terminal-shortcuts video. This page collects safe ways to find command help and work efficiently in a shell; exact lesson demonstrations were not available for verification.

Find command documentation

Command	Purpose	Example
`man COMMAND`	Open the installed manual page; press q to quit.	`man ls`
`COMMAND --help`	Show command-specific usage when supported.	`cp --help`
`apropos KEYWORD`	Search manual-page names and short descriptions.	`apropos permissions`
`type COMMAND`	Show how the shell resolves a command name.	`type cd`
`command -v COMMAND`	Print the executable path or shell resolution where available.	`command -v python3`

Manual-page sections can contain the same name for different interfaces; use `man SECTION NAME` when the target is ambiguous. Examples include `man 1 printf` for the shell command and `man 3 printf` for a C library function.

Shell history and editing

Action	Effect	Caution
Up / Down arrow	Recall earlier or later commands in the current shell history.	Review a recalled command before pressing Enter.
Tab	Complete a command or path when the shell can identify a match.	Multiple matches may require another Tab or more characters.
Ctrl+C	Send an interrupt to the foreground process.	May interrupt a command that is changing data; understand what is running.
Ctrl+L	Clear the visible terminal area in common shells.	This clears the display, not shell history or logs.
`history`	Show commands recorded in the shell history.	History can contain sensitive values; avoid putting secrets directly in commands.

Safer command habits

- Read the command and arguments before pressing Enter, especially after using history completion.
- Use `man` or `--help` to check flags before combining options.
- Quote paths and variables that may contain spaces or shell metacharacters.
- Avoid placing passwords, tokens, or other secrets in command arguments; shells may retain them in history or expose them to process listings.
- Use `clear` or Ctrl+L only to manage the display; do not mistake it for secure deletion.

Revision points

- Prefer the local manual page for distribution-specific details.
- Check a command's target and flags before running it with elevated privileges.
- Shell history and process listings can expose sensitive command-line values.

References

- NetworkChuck Linux for Hackers playlist
- Bash manual

Files and Directories

Navigation model

Linux uses a single directory tree rooted at `/`. A path beginning with `/` is absolute; a relative path is interpreted from the current directory. `.` means the current directory, `..` its parent, and `~` commonly expands to the current user's home directory.

Command	Behavior	Safe example
`pwd`	Inspect current directory.	`pwd`
`ls -la PATH`	Inspect contents, including hidden entries.	`ls -la /etc`
`cd PATH`	Change the current shell directory.	`cd "$HOME/Documents"`
`realpath PATH`	Print a resolved absolute path when available.	`realpath ./notes.txt`

File operations

Command	Effect	Example and caution
`mkdir -p DIR`	**Modify:** create a directory and missing parents.	`mkdir -p "$HOME/labs/week-1"`
`touch FILE`	**Modify:** create an empty file or update timestamps.	`touch "$HOME/labs/todo.txt"`
`cp SRC DEST`	**Modify:** copy a file.	`cp config.txt config.txt.bak`; check whether DEST already exists.
`mv SRC DEST`	**Modify:** move or rename.	`mv draft.md final.md`; verify both paths first.
`rm FILE`	**Destructive:** remove a file entry.	Do not use until the exact path and recovery needs are checked.

For file metadata, `stat FILE` shows details such as size and timestamps. `file FILE` reports the type inferred from content. These inspect a file; they do not prove that it is safe to open or execute.

Reading and locating content

Use `less FILE` for interactive reading, `head -n 20 FILE` for the first lines, and `tail -n 20 FILE` for the last lines. Search a known text file with `grep -n 'pattern' FILE`. Locate files by name with `find DIR -type f -name '*.conf'`.

Avoid searching the entire filesystem without a reason. Permission restrictions can generate errors, and broad searches may be slow or expose data outside the intended scope.

Common mistakes

- Running a relative path from the wrong working directory.
- Forgetting that dotfiles are hidden from a plain `ls` listing.
- Assuming `mv` or `cp` will always ask before replacing a destination.
- Using wildcards with modifying commands before checking what they match.

Revision points

- Confirm the working directory and target path before modifying files.
- Use inspection commands first and quote paths containing whitespace.
- Treat deletion and overwrite behavior as irreversible until recovery is tested.

References

- NetworkChuck Linux for Hackers playlist
- Coreutils command index

Users and Permissions

Identities and groups

Processes run with user and group identities. Group membership can grant access to shared files or devices. Use `whoami` to inspect the effective user, `id` to inspect numeric and named identity/group information, and `groups` to list group memberships.

Permission model

For each filesystem object, traditional Unix permissions define read (`r`), write (`w`), and execute (`x`) access for the owner, group, and others.

Object	Read	Write	Execute
Regular file	Read contents	Change contents	Run as a program/script if other requirements are met
Directory	List names	Create, remove, or rename entries (subject to other controls)	Traverse/search the directory

The meaning of directory permissions differs from file permissions. A directory may need execute permission to traverse it even when its names are visible.

Inspect and change permissions

Command	Purpose	Example
`ls -l FILE`	Inspect owner, group, and permission bits.	`ls -l notes.txt`
`stat FILE`	Inspect detailed metadata.	`stat notes.txt`
`chmod MODE FILE`	**Modify:** change permission bits.	`chmod 640 report.txt` grants owner read/write, group read, and others none.
`chown USER:GROUP FILE`	**Elevated modify:** change ownership; often requires administrator rights.	Use only under an approved change plan.
`sudo COMMAND`	Request execution with elevated privileges under local policy.	`sudo systemctl status ssh` (read-only status operation).

Numeric `chmod` values add permission bits: read=4, write=2, execute=1. `640` therefore means owner `6` (read+write), group `4` (read), others `0` (none). Confirm the intended access before changing permissions.

Least privilege

Use the narrowest account and permission set that supports the task. Avoid making files globally writable or running an interactive session as root. Elevated access increases the impact of mistakes and should be limited, auditable, and authorized.

Common mistakes

- Applying `chmod 777` as a quick fix, granting everyone read, write, and execute access.
- Confusing file write permission with permission to remove its directory entry.
- Assuming `sudo` makes a command safe or reversible.
- Changing ownership recursively without a verified target and change plan.

Revision points

- Inspect user, groups, owner, and mode before diagnosing access.
- Understand `rwx` separately for owner, group, and others.
- Make the smallest permission change needed; use elevation only when required.

References

- NetworkChuck Linux for Hackers playlist
- chmod manual
- chown manual

Processes and Services

Processes

A process is a running instance of a program. Process identifiers (PIDs) identify processes on a host. Use inspection commands to understand system state before attempting to change it.

Command	Purpose	Example
`ps -ef`	Inspect a process listing; `-e` selects all processes and `-f` uses a full-format listing.	`ps -ef`
`ps -p PID -o pid,ppid,user,stat,cmd`	Inspect selected fields for one PID.	`ps -p 1234 -o pid,ppid,user,stat,cmd`
`top`	Interactively inspect processes and resource use; press `q` to quit.	`top`
`pgrep -a NAME`	Find process IDs and command lines matching a name.	`pgrep -a ssh`

PIDs change over time and may be reused. Verify the target and owner before taking action.

Signals

`kill -TERM PID` requests graceful termination using the TERM signal. It modifies system state and can interrupt work. `kill -KILL PID` forces termination and prevents cleanup; avoid it unless a documented recovery procedure requires it. Never infer that a process is malicious from its name alone.

Services with systemd

Command	Effect	Example
`systemctl status SERVICE`	Inspect service state and recent status details.	`systemctl status ssh`
`systemctl list-units --type=service`	Inspect loaded service units.	`systemctl list-units --type=service`
`sudo systemctl restart SERVICE`	**Elevated modify:** restart a service and may interrupt users.	Use only after impact and maintenance requirements are understood.
`sudo systemctl enable SERVICE`	**Elevated modify:** configure a service to start at boot.	Make only as an approved configuration change.

Service names and service managers differ by distribution. A service being active does not establish that its configuration is secure.

Common mistakes

- Killing a process before identifying what it does and who depends on it.
- Using force termination before attempting an orderly stop.
- Restarting a production service without checking impact and change procedures.
- Treating service status as proof that an application is healthy or secure.

Revision points

- Inspect PID, owner, command, and service context before intervention.
- Prefer a graceful, documented action over a forced one.
- Separate "running" from "correctly configured" and "healthy."

References

- NetworkChuck Linux for Hackers playlist
- systemd `systemctl` manual

Package Management

The accessible playlist feed includes an episode titled "apt, dpkg, git, Python PiP (Linux Package Management)." Package commands differ across distributions. The examples below use Debian-family `apt` and `dpkg`; use the package manager documented for the host.

Inspect installed and available packages

Command	Effect	Safe example
`apt show PACKAGE`	Inspect package metadata available to configured sources.	`apt show curl`
`dpkg -l PATTERN`	Inspect locally known package status; `-l` requests a listing.	`dpkg -l 'openssh*'`
`dpkg -S PATH`	Find which installed package owns a path.	`dpkg -S /usr/bin/ssh`
`apt list --installed`	List installed packages.	Output may be lengthy; pipe to a search only when useful.

Update and install

Command	Effect	Precautions
`sudo apt update`	**Elevated modify:** refresh package indexes from configured repositories; does not itself upgrade installed packages.	Verify repository configuration and use an authorized host.
`sudo apt install PACKAGE`	**Elevated modify:** install a package and required dependencies.	Review the proposed package changes before confirming.
`sudo apt upgrade`	**Elevated modify:** upgrade installed packages according to local APT behavior.	Can change services or dependencies; follow change and maintenance procedures.
`sudo dpkg -i FILE.deb`	**Elevated modify:** install a local Debian package file.	Verify provenance and integrity; handle dependencies through the package manager.

Package operations access configured repositories and change the host. Do not install tools on systems without authorization. Avoid piping downloaded scripts directly into a privileged shell.

Python packages

Use a virtual environment for project-specific Python dependencies. This limits package changes to the project environment and helps avoid conflicts with operating-system-managed Python packages.

```
python3 -m venv .venv
. .venv/bin/activate
python -m pip install PACKAGE
python -m pip show PACKAGE
```

The first three commands create and activate a project environment and install a named package; `pip show` inspects package metadata. Package names and dependencies should be reviewed before installation. Do not use `sudo pip` as a routine workaround for permission errors.

Git basics

Git is a version-control tool, not a system package manager. The following commands inspect local repository state:

Command	Purpose
`git --version`	Show the installed Git version.
`git status`	Inspect the current repository's changes and branch state.
`git log --oneline -5`	Inspect the latest five commits in compact form.

Cloning or fetching contacts a remote and writes repository data locally. Confirm the remote source and destination path before using network operations.

Revision points

- `apt` manages repository packages on Debian-family distributions; `dpkg` works with local package database and package files.
- Refreshing package indexes is different from upgrading installed software.
- Package installation and upgrades change the host and usually require elevation.
- Keep Python project dependencies in a virtual environment; use Git to track project changes.

References

- NetworkChuck Linux for Hackers playlist
- APT manual
- dpkg manual
- Python `venv` documentation
- pip documentation

Networking Commands

Use read-only network commands first. Results depend on permissions, installed tools, host configuration, DNS, and network policy.

Command	What it inspects	Example and notes
`ip address show`	Local interfaces and assigned addresses.	`ip address show`
`ip route show`	Local routing table.	`ip route show`
`ss -tulpen`	Listening and established sockets; options select TCP/UDP, listening sockets, numeric endpoints, processes, and extended details.	`ss -tulpen`; process details may require elevated access.
`ping -c 4 HOST`	Attempts four ICMP echo requests.	`ping -c 4 example.com`; filtering or policy may prevent replies.
`getent hosts HOST`	Queries configured name-service sources for host records.	`getent hosts example.com`
`curl -I URL`	Requests response headers from a URL.	`curl -I https://example.com`; contacts the remote host and may be logged.
`traceroute HOST`	Attempts to show network hops to a host.	`traceroute example.com`; availability and results depend on the path and installed package.

Troubleshooting sequence

- 1. Inspect interface state and address configuration with `ip address show`.
- 2. Inspect the route table with `ip route show`.

- 3. Check name resolution with `getent hosts HOST`.
- 4. Test reachability only when appropriate; a missing ping reply does not prove a host is offline.
- 5. Inspect local listening sockets with `ss` and compare them with expected services.

Common mistakes and precautions

- Treating ICMP failure as proof that an application is unreachable.
- Running active probes against systems without authorization.
- Assuming a local listening socket is exposed externally; firewalls and routing affect reachability.
- Sharing outputs that contain internal addresses, usernames, or process details without review.
- Using `curl` or other clients to send requests to systems outside the approved scope.

Revision points

- Establish local interface, route, and DNS state before concluding where a failure occurs.
- Distinguish local listening state from remote reachability.
- Stay within authorized systems and record the exact host and time when troubleshooting.

References

- NetworkChuck Linux for Hackers playlist
- ip manual
- ss manual
- curl manual

Episode 1: Getting Started with Linux

Video: Linux for Hackers // EP 1 (FREE Linux course for beginners) **Source evidence:** The public video description identifies Linux, the terminal, and chapters for `pwd`, `ls`, and `cd`. **Practice level:** Safe inspection and navigation.

Learning objectives

- Describe Linux as an operating system family built around the Linux kernel and distributions.
- Open a shell and distinguish the shell prompt from commands and their output.
- Find the current directory, list entries, and move between directories.

Concepts

Linux distributions combine the kernel with system utilities, libraries, package-management tools, and applications. Kali is a Debian-based distribution oriented toward security work; its tools do not make activity authorized or safe by default. The terminal accepts commands through a shell. A command typically has a program name, options, and operands (values such as paths or hostnames).

Command	Syntax	What it does
`pwd`	`pwd`	Prints the shell's current working directory.
`ls`	`ls [OPTIONS] [PATH...]`	Lists directory entries. `-l` requests a long listing; `-a` includes dotfiles.
`cd`	`cd [PATH]`	Changes the current shell directory. `..` means parent; ~ normally means the current user's home.

Safe practice in a Kali VM

```
pwd
ls -la "$HOME"
cd "$HOME"
pwd
cd /etc
ls -la | head
```

These commands inspect locations and change only the current shell's working directory. Output depends on the VM user and installed files; no particular listing is expected. `head` limits the displayed listing to the first lines.

Common mistakes

- Typing a path as if it were a command. Use `cd PATH` to navigate.
- Confusing an absolute path (starts with `/`) with a relative path (resolved from the current directory).
- Omitting quotes around paths with spaces.
- Assuming Kali is a separate kernel or that its toolset grants permission to test another system.

Security application

Before inspecting a log, script, or configuration file, verify the current directory and target path. This reduces the chance of analyzing the wrong evidence or modifying an unintended file during later work.

Revision questions

- 1. What does `pwd` report, and does it change system state?
- 2. How do `ls -a` and `ls -l` change a directory listing?
- 3. What is the difference between `/var/log` and `../var/log`?
- 4. Why does using Kali not by itself authorize a scan?

References

- Episode 1 video and chapters
- GNU Coreutils manual
- Bash manual

Episode 2: The Linux Filesystem

Video: The Linux File System explained in 1,233 seconds // EP 2 **Source evidence:** The public title and description identify the Linux filesystem; an indexed episode summary describes the root of the tree, the "everything is a file" model, and navigation. The description says the video introduces seven command-line commands. **Practice level:** Inspect the VM filesystem; do not edit system files.

Learning objectives

- Read a Linux path from the filesystem root.
- Recognize common top-level directories and their typical purpose.
- Use file-oriented inspection commands without confusing file types with safety.

Concepts

Linux presents filesystems through one directory tree rooted at `/`. Devices and kernel interfaces may be exposed through filesystem-like paths, but not every path behaves like an ordinary disk file. Directory names such as `/etc`, `/home`, `/tmp`, `/usr`, `/var`, `/dev`, `/proc`, and `/sys` have conventional roles; exact layout and mounted filesystems vary by distribution.

Path or command	Purpose / behavior	Example
`/`	Root of the directory tree.	`ls -la /`
`/etc`	System-wide configuration, generally not personal documents.	`ls /etc`
`/home`	Home directories for regular users on many systems.	`ls /home`
`/var`	Variable data such as logs, caches, and service state.	`ls /var/log`
`file PATH`	Infers a file type from content and metadata.	`file /etc/hosts`
`stat PATH`	Shows filesystem metadata such as size, mode, and timestamps.	`stat /etc/hosts`

Safe practice in a Kali VM

```
ls -la /
ls -ld /etc /home /tmp /var
file /etc/hosts
stat /etc/hosts
```

Expected behavior: listings show entries available to the VM user; `file` reports a likely type and `stat` reports metadata. Exact values vary across systems. These commands inspect only.

Common mistakes

- Treating `/root` as the same as `/` (`/root` is commonly root's home directory).
- Assuming every file-like path stores ordinary persistent data.
- Editing a system configuration file just to explore its contents.
- Treating `file` output as a malware verdict; file-type identification is not a safety analysis.

Security application

Knowing standard locations helps an analyst find configuration, logs, user data, and device information while preserving scope. Record the path and relevant metadata before making an authorized change.

Revision questions

- 1. What does the leading `/` indicate in an absolute path?
- 2. Which kinds of information are commonly found under `/etc` and `/var`?
- 3. Why is "everything is a file" a useful model but not a literal guarantee?
- 4. Does `file sample` determine whether `sample` is safe?

References

- Episode 2 video
- Filesystem Hierarchy Standard
- GNU Coreutils manual

Episode 3: Help, Paths, and Basic File Operations

Video: HELP!! (for when you suck at Linux) // EP3 **Source evidence:** The public description presents help for command-line work. Its chapter markers cover `whoami`, `clear`, filesystem locations, `cat`, `cp`, `rm`, `adduser`, and `which`; the chapter list ranges across common directories and ends with a challenge. **Practice level:** Read-only inspection first; file changes stay in a disposable home-directory folder.

Learning objectives

- Find command documentation and identify how a command is resolved.
- Recognize common filesystem directories.
- Read and copy files safely, and understand why deletion needs care.

Commands and behavior

Command	Syntax and arguments	Effect
`whoami`	`whoami`	Prints the effective user name. Inspect only.
`command -v`	`command -v NAME`	Shows how the shell resolves a command or function. Inspect only.
`man`	`man COMMAND`	Opens the installed manual page; press q to quit.
`cat`	`cat FILE...`	Writes file contents to the terminal. Inspect only; avoid dumping secrets to a shared terminal.
`cp`	`cp SOURCE DEST`	Copies a file; modifies the destination and may overwrite depending on options.
`rm`	`rm FILE`	Removes a directory entry. Destructive; there is usually no simple undo.

The chapter list is direct evidence for these demonstrations. Use `--help` or the manual for exact option behavior because utilities and versions can differ.

Safe practice in a Kali VM

```
whoami
command -v ls
man cp
mkdir -p "$HOME/linux-practice"
printf '%s\n' 'sample note' > "$HOME/linux-practice/original.txt"
cat "$HOME/linux-practice/original.txt"
cp -n "$HOME/linux-practice/original.txt" "$HOME/linux-practice/copy.txt"
ls -l "$HOME/linux-practice"
```

The `mkdir`, redirection, and `cp` examples change files under a dedicated practice directory. `-n` asks GNU `cp` not to overwrite an existing destination. Output contents and metadata depend on the local VM. Do not practise `rm` on valuable files.

Common mistakes

- Running `cat` on binary files or sensitive logs and sending their contents to a terminal or transcript.
- Assuming `cp` always asks before replacing a destination.
- Using `rm` with wildcards or elevated privileges before checking what the pattern expands to.
- Treating `which` as universal; `command -v` also handles shell built-ins and functions.

Security application

During triage, use read-only commands to identify your account, inspect file paths, and read relevant text. Preserve original evidence; copy into an approved working location before analysis.

Revision questions

- 1. How can `man` and `--help` support safe command use?
- 2. What is the difference between `cat FILE` and `cp FILE DEST`?

- 3. Why is `rm` especially risky with a wildcard or `sudo`?
- 4. Which directories are commonly used for configuration, temporary data, and user home directories?

References

- Episode 3 video and chapters
- GNU Coreutils manual
- GNU Bash manual

Episode 4: Users, Groups, and `sudo`

Video: sudo = POWER!! (managing users in Linux) // EP4 **Source evidence:** The video description and chapter markers explicitly cover `adduser`, `/etc/passwd`, `/etc/shadow`, `useradd`, `passwd`, `usermod`, `sudo`, `su`, `visudo`, `userdel`, groups, and `gpasswd`. **Practice level:** Inspect identity only. User and privilege changes require a disposable VM and administrator authorization.

Learning objectives

- Distinguish user identity, group membership, and privileged execution.
- Locate account database files and understand why password hashes are protected.
- Explain the risk of broad `sudo` access and the purpose of `visudo`.

Commands and files

Syntax	Purpose	Impact
`whoami`; `id USER`	Inspect effective identity or a user's numeric/name IDs and groups.	Read-only
`getent passwd USER`	Query the configured account database.	Read-only; output is account metadata.
`getent group GROUP`	Query group data.	Read-only
`sudo COMMAND`	Request one command with elevated privileges under local policy.	Depends on command; often high impact
`su - USER`	Start a login shell as another user, subject to policy and credentials.	Changes the active shell identity
`sudo visudo`	Edit sudo policy with syntax validation and locking.	Privileged configuration change
`usermod -aG GROUP USER`	Append a user to supplementary group membership.	Privileged account change

`/etc/passwd` contains account records, not normally password hashes. `/etc/shadow` stores protected password authentication data and should not be exposed. Modern systems may use additional identity sources. Prefer the documented account-management tools rather than editing these files directly.

Safe practice in a Kali VM

```
whoami
id
getent passwd "$(whoami)"
sudo -l
```

The commands inspect the current user and the sudo permissions granted by local policy; `sudo -l` may request authentication. Output is host-specific and can reveal account details, so do not publish it unreviewed. Do not create users or change groups on a shared system as practice.

Common mistakes

- Confusing authentication (proving identity) with authorization (permitted actions).
- Forgetting `-a` in `usermod -aG`; without append mode, supplementary groups can be replaced.
- Giving routine accounts unnecessary administrator privileges.
- Editing sudo policy with a plain editor rather than the validation-capable `visudo`.
- Assuming a successful `sudo` command makes the underlying action safe.

Security application

Use unique accounts, least privilege, limited elevation, and auditable changes. Review group membership and sudo rules after role changes or suspected account compromise.

Revision questions

- 1. What question does authentication answer? What question does authorization answer?
- 2. What information is normally in `/etc/passwd` versus `/etc/shadow`?
- 3. Why is `usermod -aG` different from `usermod -G`?
- 4. Why should sudoers changes be made through `visudo`?

References

- Episode 4 video and chapters
- `usermod` manual
- `sudoers` manual

Episode 5: Linux Package Management

Video: apt, dpkg, git, Python PiP (Linux Package Management) // EP 5 **Source evidence:** The public description and chapter markers cover Debian packages, `dpkg`, APT repositories, `apt update`, installed packages, removal, upgrades, `aptitude`, Snap, Python `pip`, RubyGems, Git, `git clone`, and requirements files. **Practice level:** Inspection examples are safe; installations and upgrades modify the VM and use elevated access.

Learning objectives

- Distinguish a package archive from a repository-aware package manager.
- Check package source and proposed system changes before installation.
- Keep Python project dependencies separate from OS packages where possible.

Command reference

Syntax	Purpose	Change level
`apt show PACKAGE`	Inspect package metadata from configured sources.	Read-only
`dpkg -l PATTERN`	List local package database entries matching a pattern.	Read-only
`sudo apt update`	Refresh local package indexes; does not itself upgrade installed packages.	Elevated; writes index data and contacts repositories
`sudo apt install PACKAGE`	Install a package and dependencies from configured sources.	Elevated system change
`sudo dpkg -i FILE.deb`	Install a local Debian package archive.	Elevated system change; dependencies may need repair

Syntax	Purpose	Change level
`git clone URL DIR`	Copy a remote repository into a new local directory.	Network and local filesystem change
`python3 -m venv .venv`	Create a project-local Python environment.	Local filesystem change
`python -m pip install -r requirements.txt`	Install dependencies listed in a project file into the active environment.	Package installation; review sources and versions first

Safe practice in a Kali VM

```
apt show curl
dpkg -l 'curl'
python3 -m venv "$HOME/linux-practice/.venv"
```

The first two commands inspect package information; the final command creates an isolated environment in the practice folder. To install software, first verify the repository configuration and review APT's proposed actions. Never use `sudo pip` as a routine fix for an environment conflict.

Common mistakes

- Treating `apt update` as an upgrade; it refreshes repository indexes.
- Downloading and installing a `.deb` from an unverified source.
- Running a repository's dependency file without reviewing package names and origins.
- Mixing project Python packages with distribution-managed Python packages.
- Cloning an untrusted repository and executing its install script without review.

Security application

Package management is part of patching and software provenance. Use trusted repositories, minimize installed software, review updates for service impact, and preserve enough version information to investigate changes.

Revision questions

- 1. What does `apt update` change, and what does it not do?
- 2. How does `dpkg -i` differ from `apt install`?
- 3. Why use a virtual environment for a Python project?
- 4. What should be checked before cloning and running code from a repository?

References

- Episode 5 video and chapters
- APT manual
- dpkg manual
- Python `venv`

Episode 6: Services and `systemd`

Video: Start, stop, restart Linux services (daemon HUNTING!!) // EP 6 **Source evidence:** The indexed description says the episode covers starting, stopping, restarting, and enabling services. Its chapter markers identify daemons, systemd, `systemctl`, service status, boot enablement, hunting services, and troubleshooting. **Practice level:** Status and listing are inspection; start/stop/restart/enable are state changes.

Learning objectives

- Explain the difference between a process, a daemon, a service, and a systemd unit.

- Inspect service state before taking action.
- Distinguish running now (`start`) from starting during boot (`enable`).

Command reference

Syntax	Behavior	Impact
`systemctl status UNIT`	Show unit state and recent status information.	Inspect
`systemctl list-units --type=service`	List currently loaded service units.	Inspect
`sudo systemctl start UNIT`	Start a service now.	State change; can open a listener or consume resources
`sudo systemctl stop UNIT`	Stop a service now.	State change; can interrupt dependent work
`sudo systemctl restart UNIT`	Stop and start a service.	State change; brief outage or lost sessions possible
`sudo systemctl enable UNIT`	Configure a unit to start automatically at boot.	Persistent configuration change
`systemctl is-enabled UNIT`	Inspect boot enablement state.	Inspect

Safe practice in a Kali VM

```
systemctl status ssh
systemctl list-units --type=service --state=running
systemctl is-enabled ssh
```

These commands inspect local units; output differs between Kali installations. Do not enable or expose a remote-access service merely to practise. If testing state changes, snapshot the VM and use a service you understand, then restore the original state.

Common mistakes

- Assuming a service named `ssh` exists or uses the same unit name on every distribution.
- Confusing `start` with `enable`.
- Restarting a service without considering users, dependencies, logs, or recovery.
- Treating an "active" status as proof that the service is correctly configured or externally reachable.

Security application

Inventory active services and compare them with an approved baseline. Disable unnecessary services only through a controlled change; confirm listening sockets and firewall exposure separately.

Revision questions

- 1. What is a daemon, and how does it relate to a service unit?
- 2. Which `systemctl` operation affects the current session? Which affects future boots?
- 3. Why might `systemctl status` succeed while the service is still insecure?
- 4. What should be checked before stopping a production service?

References

- Episode 6 video and chapters
- `systemctl` manual
- systemd unit concepts

Episode 7: Process Management

Video: KILL Linux processes!! (also manage them) // EP 7 **Source evidence:** The public description states that the episode covers finding processes, moving them between foreground and background, and terminating them. **Practice level:** Listing is inspection; signals and job-control actions alter process state.

Learning objectives

- Identify a process by PID, command, owner, and parent process.
- Understand foreground/background jobs and signals.
- Prefer graceful termination and verify the target before sending a signal.

Command reference

Syntax	Purpose	Impact
`ps -ef`	List processes; `-e` selects all, `-f` uses full format.	Inspect
`pgrep -a NAME`	Find matching PIDs and command lines.	Inspect
`jobs -l`	List jobs started by the current shell.	Inspect
`COMMAND &`	Start a command asynchronously in the shell.	Starts a process
`fg %N`; `bg %N`	Bring a shell job to foreground or continue it in background.	Alters job state
`kill -TERM PID`	Request orderly termination with TERM.	May interrupt work
`kill -KILL PID`	Force immediate termination; process cannot handle cleanup.	High impact; last resort

Safe practice in a Kali VM

```
sleep 300 &
jobs -l
pgrep -a sleep
```

This starts a harmless timer in the current VM. To stop only the timer you started, identify its PID from `jobs -l`, then use `kill -TERM PID` and verify it exited. The literal `PID` is a placeholder: replace it with the verified numeric process ID. Do not copy a guessed PID.

Common mistakes

- Sending a signal based only on a process name; multiple unrelated processes may match.
- Using `kill -9` before trying graceful termination.
- Forgetting PIDs can be reused after a process exits.
- Confusing shell jobs with all system processes.

Security application

During investigation, preserve process context: executable path, owner, parent, arguments, start time, and related network connections. In an incident, coordinate termination with evidence-preservation and containment procedures.

Revision questions

- 1. What fields help you confirm that a process is the intended target?
- 2. How does `TERM` differ from `KILL`?
- 3. What does `&` do in a shell command?

- 4. Why should an analyst collect process context before terminating a suspicious process?

References

- Episode 7 video
- `ps` manual
- `kill` manual

Episode 8: A Local Web Server and `curl`

Video: 2 cool (web) things to do in Linux // EP 8 **Source evidence:** The public description says the episode demonstrates managing web services with `curl`. Chapter markers cover starting a one-command website, changing directories and ports, making requests from the command line, and inspecting request headers. **Practice level:** Bind the test server to loopback and keep requests local.

Learning objectives

- Start a temporary HTTP server for local learning.
- Understand that a server listens on an address and port.
- Use `curl` to make an HTTP request and inspect response or request metadata.

Commands

Syntax	Purpose and arguments	Impact
`python3 -m http.server PORT --bind 127.0.0.1 --directory DIR`	Serve files from `DIR` on a local test port.	Starts a network listener, restricted here to loopback
`curl URL`	Send an HTTP request and print the response body.	Contacts the target; responses may be logged
`curl -I URL`	Request headers only where supported by the server.	Network request
`curl -v URL`	Show connection and protocol details, including request/response headers.	May expose sensitive headers; use only on lab traffic

Safe practice in a Kali VM

Terminal 1:

```
mkdir -p "$HOME/web-lab"
printf '%s\n' 'Local practice page' > "$HOME/web-lab/index.html"
python3 -m http.server 8000 --bind 127.0.0.1 --directory "$HOME/web-lab"
```

Terminal 2:

```
curl -i http://127.0.0.1:8000/
```

Expected behavior: the second terminal displays an HTTP status line, response headers, and the page body; the server logs a request in the first terminal. This is a documented expected behavior of the commands, not a captured test result. Stop the server with Ctrl+C when finished.

Common mistakes

- Binding to all interfaces when only local testing is needed.
- Treating a port number as proof of which application is listening.
- Confusing request headers sent by a client with response headers returned by a server.
- Following redirects, submitting data, or using verbose output without considering sensitive content.

Security application

Use `curl` to inspect HTTP behavior in an authorized local lab, verify expected headers, and troubleshoot a service. Keep test services bound to loopback unless an isolated network lab explicitly requires other hosts to connect.

Revision questions

- 1. What is the difference between `127.0.0.1` and a wildcard bind address?
- 2. Which option asks for response headers only? Which provides verbose protocol details?
- 3. What does a listening port tell you, and what does it not tell you?
- 4. How do you stop the temporary Python server?

References

- Episode 8 video and chapters
- Python `http.server` documentation
- curl manual

Episode 9: Terminal Efficiency

Video: you need to HACK faster!! (Linux Terminal hacks YOU NEED!!) **Source evidence:** The public description focuses on navigating the Linux terminal quickly. Indexed chapter text refers to Bash startup, navigation shortcuts, command-line editing, and log-oriented workflow. Exact details are not equally available for every segment, so this note avoids attributing a fixed shortcut list to the video. **Practice level:** Shell interaction; review recalled commands before execution.

Learning objectives

- Use shell history, completion, and line-editing shortcuts to reduce repetitive typing.
- Treat speed as secondary to checking the command and its target.
- Recognize that command history and terminal scrollback can contain sensitive data.

Useful Bash workflow

Action	Typical Bash behavior	Security note
Tab	Complete a command or path where unambiguous.	Review the completed path before running a modifying command.
Up / Down	Move through recent history entries.	Do not press Enter on an unreviewed recalled command.
`history`	Display command history for the current shell.	May expose secrets accidentally typed as arguments.
Ctrl+A / Ctrl+E	Move to beginning/end of the current command line in common Bash editing modes.	Inspect edited command in full before execution.
Ctrl+R	Search history interactively in common Bash configuration.	A matching command may be stale or unsafe.
`clear`	Clear the visible terminal display.	Does not erase shell history, logs, or evidence.

Shortcuts can vary with shell, terminal, and key bindings. Check the shell's local documentation and configuration.

Safe practice in a Kali VM

```
printf '%s\n' 'one' 'two' 'three' > "$HOME/terminal-practice.txt"
grep -n 'two' "$HOME/terminal-practice.txt"
history | tail -n 5
```

The first line creates or overwrites a file in the home directory, so confirm that exact path first. The final command displays only recent history lines, which can still contain sensitive values. Expected behavior: `grep` reports the matching line and line number; actual output depends on the file contents.

Common mistakes

- Optimizing for keystrokes before understanding what the command does.
- Re-running a history entry that has the wrong working directory or target.
- Assuming `clear` removes sensitive terminal data from history or logs.
- Putting passwords, tokens, or API keys in command arguments or shell history.

Security application

Efficient command-line work is useful for reviewing logs and navigating incident artifacts, but reproducibility and auditability matter more than speed. Use explicit paths, record important actions, and keep secrets out of commands.

Revision questions

- 1. Why can a history shortcut create risk even if it saves time?
- 2. Does `clear` securely erase shell history?
- 3. How can Tab completion reduce path errors, and why must the result still be reviewed?
- 4. What information should not be entered directly as a command argument?

References

- Episode 9 video
- Bash manual: command line editing
- Bash manual

Episode 10: File Operations and Failure Modes

Video: i KILLED my Linux computer!! (to teach you something) **Source evidence:** The public description says the episode demonstrates breaking a Linux system and creating a very large number of directories. Indexed chapter markers name `echo`, `mkdir`, `mv`, and `cp`; the video is part of the playlist. These examples are deliberately treated as hazardous demonstrations, not recommended practice. **Practice level:** Safe file operations only, inside a disposable VM snapshot and a dedicated temporary directory.

Learning objectives

- Understand how simple file and directory operations can have a large effect when paths, loops, or volume are uncontrolled.
- Distinguish creating, copying, moving, overwriting, and deleting data.
- Practise only bounded operations in a directory created for the exercise.

Commands identified by the public chapters

Syntax	Purpose	Risk
`echo TEXT > FILE`	Writes text through shell redirection. > creates or truncates the destination.	Overwrites an existing file.

Syntax	Purpose	Risk
`mkdir DIR`	Creates one directory.	Low if the path is deliberate; repeated creation can consume storage/inodes.
`mv SOURCE DEST`	Moves or renames a path.	May replace a destination or make files harder to locate.
`cp SOURCE DEST`	Copies a file or, with options, directory trees.	Can overwrite data and consume storage.
`rm FILE`	Removes a directory entry.	Destructive; recovery may be difficult.

The video title and description explicitly frame the system-damage demonstration; the original high-impact command is intentionally not reproduced here. Consult each utility's manual before using options not shown in this safe exercise.

Bounded practice in a disposable Kali VM

```
lab="$HOME/file-ops-practice"
mkdir -p "$lab"
printf '%s\n' 'practice only' > "$lab/original.txt"
cp -n "$lab/original.txt" "$lab/copy.txt"
mkdir "$lab/one-directory"
ls -la "$lab"
```

This creates two small text files and one directory under a fixed path in the home folder. `ls` lets you inspect the result. No output is claimed as a captured run. Avoid loops, recursive operations, broad wildcards, root-owned locations, and commands copied from a video until you understand their full effect.

Common mistakes and precautions

- Assuming `>` appends; it truncates. Use `>>` only when appending is intended.
- Running commands from an unexpected working directory.
- Using `*` or recursive flags without listing the matched paths first.
- Creating unbounded numbers of files or directories; storage or inode exhaustion can make a VM unusable.
- Treating `sudo` as a safety mechanism. It increases the scope of a mistake.

Security application

File integrity and availability depend on controlling write and deletion scope. Use least privilege, backups or VM snapshots, explicit paths, and reviewable changes. For forensic work, preserve originals and document any necessary copy or transformation.

Revision questions

- 1. What is the difference between `>` and `>>`?
- 2. Why can a loop that creates many directories affect system availability?
- 3. What should you verify before using `mv`, `cp`, or `rm`?
- 4. Why is a disposable VM snapshot useful for learning hazardous commands?

References

- Episode 10 video and chapter information
- GNU Coreutils manual
- Bash manual: redirections

Final Revision Checklist

Before a command

- Confirm the working directory (`pwd`), target path, current account (`id`), and command syntax (`man` or `--help`).
- Classify the command: inspect, local file change, network request, privileged change, or destructive action.
- Quote paths and variables; review shell history before re-running a command.
- Practise in a disposable Kali VM snapshot with no valuable data.

Core Linux ideas

- `/` is the filesystem root; `/root` is commonly the root user's home. Paths and mounted filesystems vary by distribution.
- `>` truncates a destination; `>>` appends. A pipe passes standard output to another command.
- `apt update` refreshes repository metadata; it does not upgrade installed packages.
- `start` affects current service state; `enable` affects boot configuration.
- Identify a process by PID, owner, and command before signalling it. Prefer graceful termination to forced termination.
- A listener is local state; routing and firewall policy affect whether other hosts can reach it.
- `clear` hides current display content but does not erase history or logs.
- `sudo` expands impact and does not make a command safe.

Safe file-operation checklist

Before `cp`, `mv`, redirection, or removal, verify the exact source, destination, wildcard expansion, privilege level, and recovery plan. Avoid recursive or unbounded operations while learning. Keep demonstration services bound to `127.0.0.1` unless an isolated lab requires otherwise.

Review prompts

- 1. How would you inspect an unfamiliar service without changing its state?
- 2. What is the difference between an operating-system package and a Python project dependency?
- 3. Why can a process name alone be an unsafe termination target?
- 4. How can a command-line shortcut make work faster but less safe?